AI-Native B2B SaaS Platform: Comprehensive Architecture & PRD

This document serves as the comprehensive Product Requirements Document (PRD) and technical specification for the dynamic, AI-native B2B e-commerce and booking platform. This specification is designed to be ingested by code generation LLMs (like Claude) to bootstrap the NestJS backends.

1. System Architecture Overview

The system is split into two specialized NestJS/Node.js monoliths, decoupled by an AWS SQS FIFO queue to isolate long-running, non-deterministic AI processes from fast, deterministic database transactions.

- **MarketplaceEngine (The Muscle):** The deterministic core application runtime. Handles CRUD operations, routing, dynamic data execution, and tenant isolation via DynamoDB.
- **MarketplaceAgents (The Brain):** The AI orchestration layer. Utilizes LangGraph.js for multi-turn conversational intent extraction, JSON blueprint generation, and strict schema validation.

2. MarketplaceEngine (The Backend Engine)

2.1 Database Strategy

The backend utilizes AWS DynamoDB operating on a **Single-Table Design** (Table: MarketplaceEngine). Data is strictly isolated across tenants using the Partition Key.

- **Partition Key (PK):** TENANT#<client_id>
- **Sort Key (SK):** Defines the entity type (e.g., PRODUCT#<id>, ORDER#<id>)

The system adopts a **Hybrid Schema Pattern**. Every record contains a strict set of typed core fields for primary access patterns, alongside a schema-less attributes JSON map (Map in DynamoDB) that absorbs all AI-generated dynamic fields.

2.2 Core Entity Schemas (TypeScript)

```
export type ProductStatus = 'active' | 'draft' | 'archived' |
'out_of_stock';

// 1. PRODUCT SCHEMA
export interface ProductCore {
  PK: string; // TENANT#<client_id>
  SK: string; // PRODUCT#<product_id>
  entityType: 'Product';

  id: string;
  status: ProductStatus;
```

```
  name: string;
  description: string;
  price: number;
  category: string;
  stock: number;
  images: string[];
  createdAt: string;
  updatedAt: string;

  attributes: Record<string, any>; // AI Dynamic Extension
}

// 2. ORDER SCHEMA
export interface OrderCore {
  PK: string;
  SK: string; // ORDER#<order_id>
  entityType: 'Order';

  id: string;
  userId: string;
  status: string;
  totalAmount: number;
  items: Array<{ productId: string; quantity: number; unitPrice:
number; }>;
  shippingAddress: Record<string, string>;
  createdAt: string;

  attributes: Record<string, any>;
}

// 3. EVENT & BOOKING SCHEMAS
export interface EventCore {
  PK: string; SK: string; entityType: 'Event';
  id: string; title: string; capacity: number;
  startDate: string; endDate: string; basePrice: number;
  attributes: Record<string, any>;
}

export interface BookingCore {
  PK: string; SK: string; entityType: 'Booking';
  id: string; eventId: string; userId: string;
  status: string; ticketQuantity: number;
  attributes: Record<string, any>;
}

// 4. FLOW SCHEMA (Dynamic Schema Audit Trail)
export interface FlowCore {
  PK: string; SK: string; entityType: 'Flow';
```

```
  id: string; targetEntity: string; actionType: string;
  status: string; blueprintDiff: Record<string, any>;
  createdAt: string;
}
```

2.3 RESTful API Specifications

All endpoints exist under /api/v1. Tenant context is implicitly derived via JWT Authorization headers.

Module	Endpoints	Notes
Products	GET /products POST /products POST /products/search	Search endpoint accepts JSON bodies for filtering nested dynamic attributes.
Orders	GET /orders POST /orders	POST route locks in unitPrice historically and decrements product stock.
Events	GET /events POST /events POST /bookings	Bookings must utilize DynamoDB transactional writes to enforce strict capacity limits.
Flows	POST /flows POST /flows/:id/execute	Execute route triggers background backfill for historical data mapping.

3. MarketplaceAgents (The AI Orchestrator)

3.1 LangGraph State Machine

The multi-turn conversational agent tracks workflow phases and persists conversational memory via a Redis checkpointer. The graph halts execution when human feedback is required.

```
export type AgentPhase = 'gathering' | 'drafting' | 'validating' |
'awaiting_confirmation' | 'done';

export interface AgentState {
  tenantId: string;
  messages: BaseMessage[]; // Chat history
  phase: AgentPhase;
  targetEntity: 'Product' | 'Order' | 'Event' | 'Booking' | 'Flow' |
null;
  draftPayload: any | null;
  missingRequirements: string[];
}
```

3.2 Graph Nodes & Flow Logic

1. **analyzeIntentNode (LLM):** Parses user messages to determine target entity and the

requested action.

2. **requirementCheckNode (LLM):** Evaluates if all necessary specs are present. If missing, transitions phase to gathering and halts to wait for user input.
3. **draftSchemaNode (LLM):** Constructs the JSON blueprint mapping to the Engine's dynamic extension attributes.
4. **validationNode (Deterministic Code):** Strict Zod/TypeScript validation of the drafted JSON. Fails loop back to drafting silently. Passes transition to awaiting_confirmation and halt.
5. **dispatchNode (Deterministic Code):** Packages the valid blueprint and pushes to AWS SQS.

4. Inter-Service Communication Contract

MarketplaceAgents communicates output to MarketplaceEngine exclusively via an **AWS SQS FIFO Queue** (e.g., marketplace-engine-jobs.fifo). The MessageGroupId is strictly set to the tenantId to guarantee execution order and prevent race conditions for concurrent schema updates.

```
{
  "MessageGroupId": "TENANT#client_123",
  "MessageBody": {
    "action": "PROPOSE_FLOW",
    "targetEntity": "Event",
    "tenantId": "TENANT#client_123",
    "payload": {
      "addedFields": [
        { "name": "dietaryNeeds", "type": "string" }
      ]
    },
    "metadata": {
      "threadId": "thread_88291"
    }
  }
}
```